Instituto Superior de Engenharia

Politécnico de Coimbra

SafetySec

Arquiteturas Móveis - 2025/2026

Trabalho prático - SafetYSec

Henrique Bernardi Netto a2023135569@isec.pt

Tiago Filipe dos Santos Morais a2023135789@isec.pt

1.Introdução **3**
2. Ecrãs **4**
2.1. Login/Registo 4
2.2. Home 5
2.3. Definições 5
2.4. Protegido 6
2.4.1 Dashboard 6
2.4.2 Gerenciador de monitores 7
2.4.3 Regras do protegido 8
2.5. Monitor 9
2.5.1 Dashboard 9
2.5.2 Regras do protegido 10
2.5.3 Criar uma nova regra 10
2.5.4 Criar uma nova regra 11
2.5.5 Alerta emitido por um protegido 12
3. Arquitetura da Solução **13**
3.1. Padrão Arquitetural (MVVM) 13
3.2. Funcionalidades e Implementação 13
4. Detalhes de Implementação e Soluções Técnicas **15**
4.1. Serviço de Monitorização em Segundo Plano 15
4.1.1 Deteção de Quedas e Acidentes 15
4.1.2 Geofencing e Velocidade 16
4.1.3 Verificação de Janelas Temporais 18
4.1.4 Deteção de Inatividade 18
4.2. Captura de Prova em Vídeo (CameraX)
Para garantir a recolha de evidências forenses após a deteção de um evento crítico, a aplicação integra a API CameraX do Android Jetpack, cuja lógica se encontra encapsulada na classe utilitária VideoManager. 19
4.3. Associação Segura e Autenticação 19
4.4. Gestão de Alertas e Notificações 20
5. Estrutura de Dados (Firestore) **21**
6. Interface e Usabilidade (UI/UX) **23**
7. Conclusão e Trabalho Futuro **24**

1.Introdução

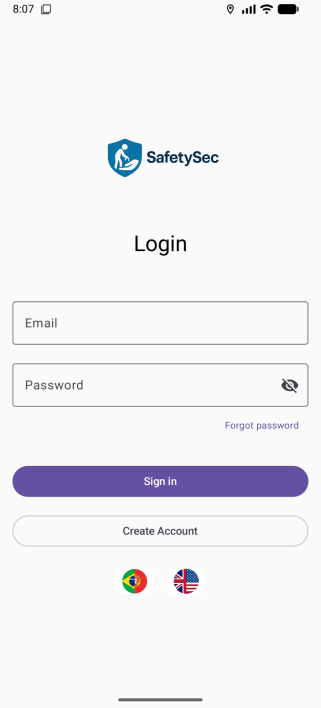
O projeto **SafetySec** foi concebido no âmbito da unidade curricular de Arquiteturas Móveis com o objetivo de desenvolver uma aplicação móvel robusta e fiável destinada à **monitorização de utilizadores vulneráveis**, designados por *Protegidos*. A aplicação permite que utilizadores responsáveis, designados por *Monitores*, recebam **alertas em tempo real** sempre que seja detectada uma situação potencialmente perigosa.

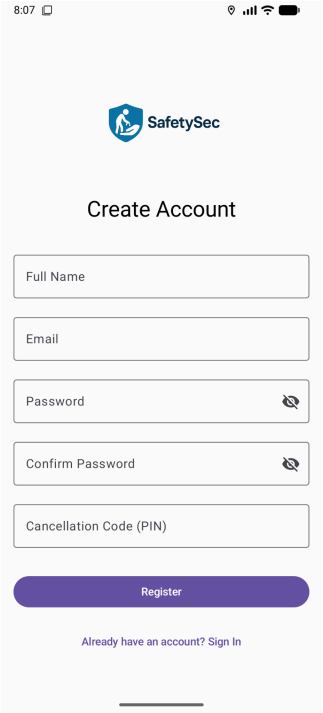
A solução centra-se na **deteção automática de incidentes**, tais como quedas, acidentes e violações de zonas seguras (*geofencing*), recorrendo aos sensores nativos do dispositivo móvel. Paralelamente, integra mecanismos de **segurança ativa**, nomeadamente a gravação automática de vídeo após um evento crítico e a utilização de **autenticação forte (MFA)**, garantindo elevados níveis de fiabilidade, privacidade e proteção dos dados.

O principal objetivo do projeto foi aplicar boas práticas de arquitetura móvel moderna, explorando tecnologias atuais do ecossistema Android, assegurando simultaneamente escalabilidade, manutenibilidade e uma experiência de utilização acessível.

2. Ecrãs

2.1. Login/Registo





Início de Sessão (LoginScreen):
Permite a autenticação através de e-mail e palavra-passe. A lógica interage com o `AuthViewModel` para validar as credenciais.

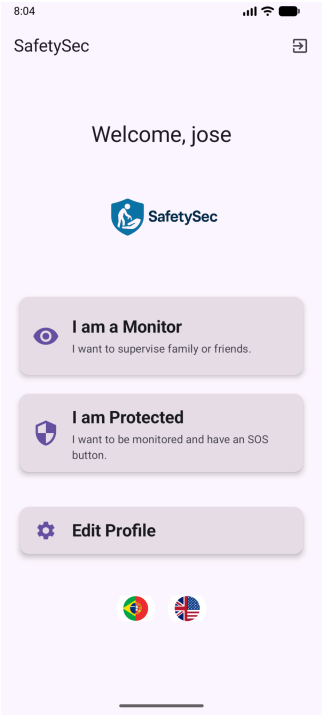
Inclui ainda um seletor de idioma (*LanguageSelector*) para alternar entre Português e Inglês e um link para recuperação de palavra-passe ("Esqueceu-se da palavra-passe?").

MFA (MfaVerificationScreen):
Ecrã intermédio de segurança. Simula o envio de um código (visível no *Logcat* do Android Studio para efeitos de desenvolvimento e teste) que o utilizador deve introduzir para concluir o início de sessão.

Registo (RegisterScreen):
Para além dos dados habituais de identificação (Nome, E-mail, Palavra-passe), este ecrã obriga à definição de um Código de Cancelamento (PIN de 4 dígitos).

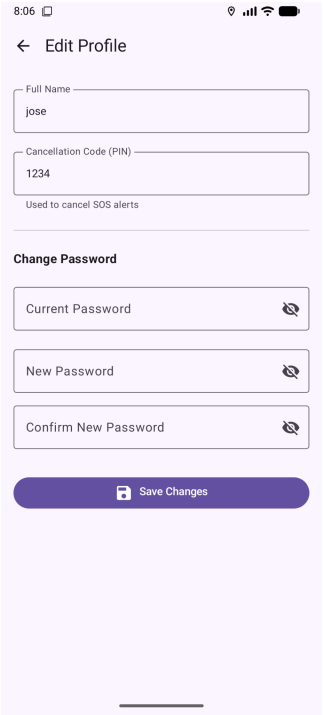
Este código é armazenado no perfil do utilizador (`SafetyUser`) e é fundamental para a arquitetura de segurança, pois constitui o único mecanismo que permite ao utilizador Protegido abortar um pedido de socorro (SOS) ou cancelar um alerta ativo antes de os Monitores tomarem medidas de emergência.

2.2. Home



Funciona como o “hub” central de navegação após o início de sessão. Verifica o estado atual do utilizador (por exemplo, se já definiu um perfil). Apresenta dois grandes cartões de seleção de modo: **“Sou Monitor”** e **“Sou Protegido”**.
Definições em que o utilizador pode alterar os seus dados, também contém a opção **Terminar Sessão**, permitindo trocar de conta ou reconfigurar o perfil de forma simples.

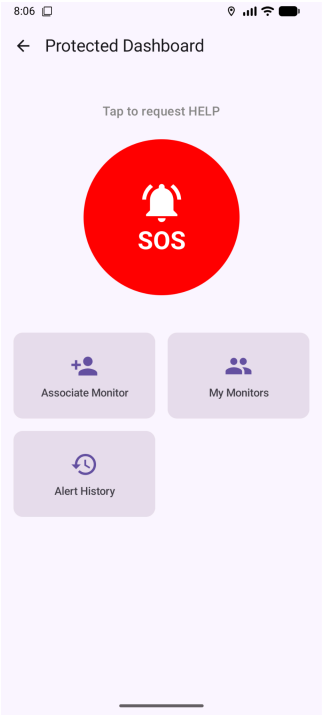
2.3. Definições



Definições (EditProfileScreen):
Ecrã de gestão da conta que permite a atualização de dados sensíveis. O utilizador pode alterar o nome de apresentação e o seu **Código de Cancelamento (PIN)**.
Inclui uma secção dedicada à segurança (**"Alterar Palavra-passe"**), que exige a validação da palavra-passe atual antes de permitir a definição de uma nova, garantindo que as credenciais não podem ser alteradas por terceiros que encontrem o dispositivo desbloqueado.

2.4. Protegido

2.4.1 Dashboard



Dashboard (ProtectedDashboardScreen):
Interface principal para o utilizador vulnerável. O elemento central é o **Botão SOS**, concebido para ser acionado rapidamente em situações de pânico. Ao ser premido, inicia uma rotina no `SafetyMonitoringService` que capta a localização e grava um vídeo em segundo plano após 10 segundos.

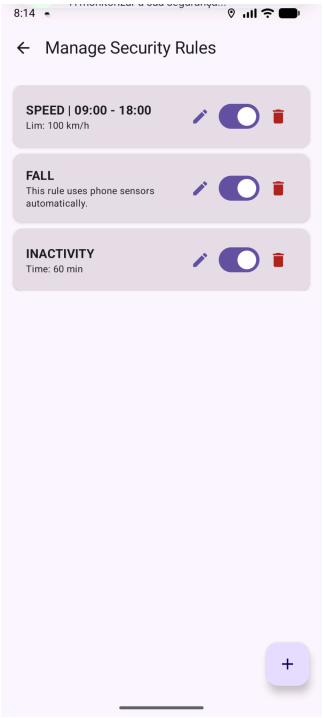
Associação:
Inclui um botão **"Associar Monitor"**, que comunica com o `AssociationViewModel` para gerar um código alfanumérico aleatório de 6 dígitos (válido por 5 minutos).

Permissões:
O ecrã verifica proativamente as permissões do sistema (Câmara, Localização precisa/aproximada) e solicita-as caso não estejam concedidas, uma vez que são essenciais para o funcionamento dos alertas.

2.4.2 Gerenciador de monitores

Os Meus Monitores (MyMonitorsScreen):
Lista todos os utilizadores que têm permissão para monitorizar este protegido. A informação é obtida em tempo real a partir do Firestore.
Cada item da lista inclui um botão de **Remover**, permitindo ao Protegido revogar unilateralmente o acesso de um monitor específico a qualquer momento, garantindo o controlo total sobre a sua privacidade.
Também possui um botão de **Configuração,** em que é possível editar/ligar/desligar todas as regras.

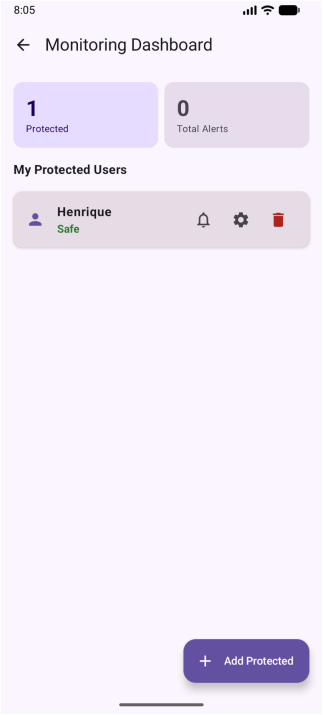
2.4.3 Regras do protegido



Regras Ativas (ManageRulesScreen – Modo de Leitura):
Embora a criação de regras seja da responsabilidade do Monitor, o Protegido pode aceder a este ecrã para visualizar as restrições de segurança que lhe estão aplicadas.
Esta funcionalidade oferece transparência ao utilizador relativamente ao que está a ser monitorizado.

2.5. Monitor

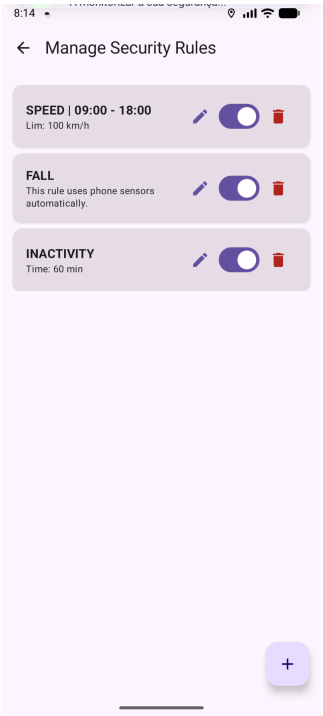
2.5.1 Dashboard



Dashboard do Monitor (MonitorDashboardScreen):
Painel de controlo que lista todos os utilizadores Protegidos associados a este Monitor. Cada “cartão” de utilizador apresenta o estado atual (Seguro / Em Perigo), possui um botão para ver as notificações, um para gerenciar as regras e por fim um para eliminar o protegido.

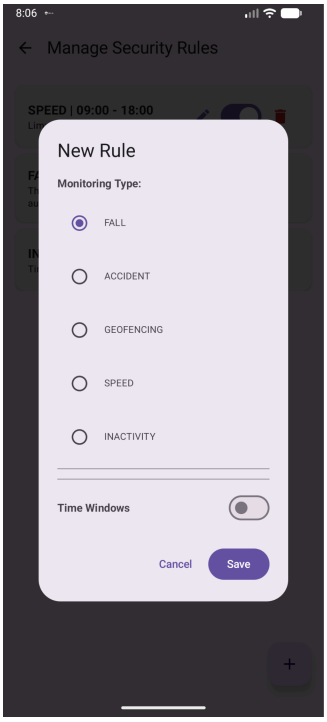
Adicionar Protegido:
Inclui um botão flutuante (*FAB*) que abre um diálogo para introdução do código de 6 dígitos fornecido por um utilizador Protegido, criando a associação na base de dados através do `AssociationRepository`.

2.5.2 Regras do protegido



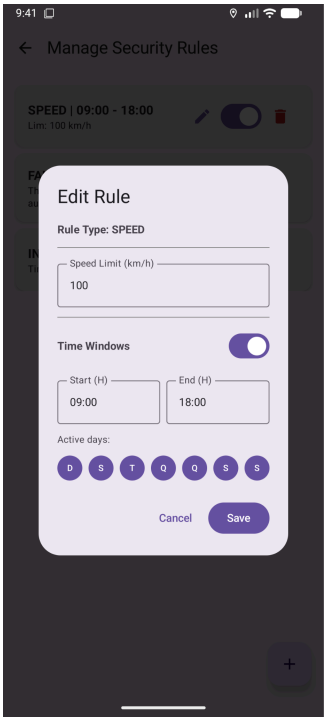
Gestão de Regras (ManageRulesScreen – Lista):
Lista todas as regras criadas para um utilizador específico. Permite ao Monitor ativar ou desativar regras individualmente sem necessidade de as eliminar (através de interruptores ou caixas de seleção), facilitando o ajuste da monitorização à medida que a rotina do Protegido se altera.

2.5.3 Criar uma nova regra



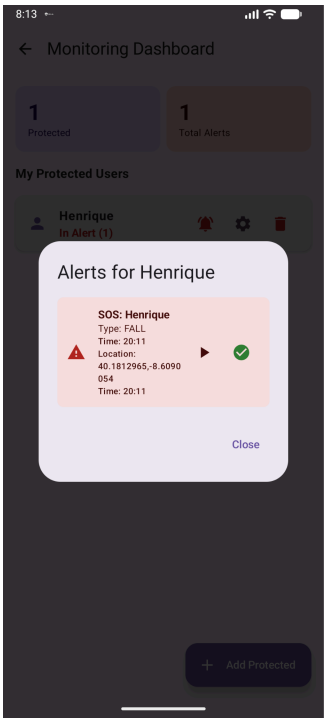
Configuração de Regras (ManageRulesScreen):
Permite ao Monitor definir parâmetros de segurança proativos para um Protegido específico. O Monitor seleciona o **Tipo de Regra** e de acordo com a regra selecionada pede um determinado parâmetro, para todas as regras é possível inserir uma janela de monitoramento.

2.5.4 Criar uma nova regra



Contexto da Regra (ManageRulesScreen – continuação):
Permite configurar os **Valores** e **Horários de Atividade** (hora de início e de fim) e os dias da semana, garantindo que a regra (por exemplo, “Estar na escola”) apenas gera alertas nos períodos relevantes.

2.5.5 Alerta emitido por um protegido



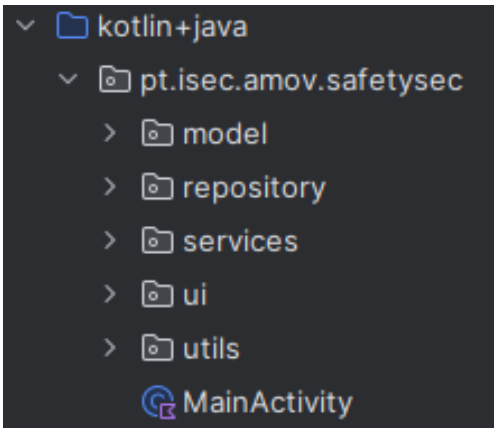
Visualização de Alerta (AlertActivity):
Este ecrã é aberto automaticamente (através de notificação ou do `MonitorService`) quando um utilizador Protegido entra em estado de emergência.

Funcionalidades:
Apresenta um mapa com um marcador na localização exata do incidente e um leitor de vídeo para reprodução da gravação enviada pelo dispositivo do Protegido (quando disponível).

Resolução:
Permite ao Monitor marcar o alerta como **"Resolvido"**, notificando o sistema de que a situação foi controlada.

3. Arquitetura da Solução

3.1. Padrão Arquitetural (MVVM)



A aplicação SafetySec foi desenvolvida seguindo rigorosamente o padrão **Model–View–ViewModel (MVVM)**, promovendo uma clara separação de responsabilidades e facilitando a evolução e manutenção do código.

- **View (Interface do Utilizador)**
  Implementada integralmente com **Jetpack Compose**, a camada de apresentação é responsável apenas pela renderização do estado da aplicação. Atividades como **`MainActivity`** e ecrãs como **`MonitorDashboardScreen`** limitam-se a observar estados expostos pelos *ViewModels*, reagindo de forma declarativa às alterações, sem conter lógica de negócio.

- **ViewModel**
  Classes como **`DashboardViewModel`** e **`AuthViewModel`** atuam como intermediárias entre a UI e a lógica da aplicação. Estas gerem o estado da interface através de `StateFlow` e `MutableState`, garantindo persistência de estado durante alterações de configuração, como rotações de ecrã.

- **Model / Repository**
  Esta camada encapsula o acesso a dados e a lógica de baixo nível, incluindo serviços como **`SafetyMonitoringService`** e repositórios como **`AuthRepository`**. É responsável pela comunicação com o Firebase Firestore, bem como pela interação direta com sensores e serviços do sistema.

3.2. Funcionalidades e Implementação

- **Backend:** Firebase
  - Authentication - Para registo/login
  - Firestore Database - Para armazenar informação.
  - Firebase Storage - Para armazenar os vídeos
- **Concorrência:** Kotlin Coroutines e Flow para operações assíncronas e reativas

- **Sensores e Multimédia:**
    - `SensorManager` (acelerómetro)
    - `FusedLocationProviderClient` (localização)
    - CameraX (captura de vídeo)

4. Detalhes de Implementação e Soluções Técnicas

4.1. Serviço de Monitorização em Segundo Plano

A implementação do `SafetyMonitoringService` recorre à definição de tipos específicos de serviço em primeiro plano no manifesto (`foregroundServiceType="location|dataSync"`), garantindo a conformidade com as políticas rigorosas do Android 14.

Para a gestão eficiente de recursos, o serviço implementa lógica de **Cooldown**:

- **Prevenção de Falsos Positivos:** Foi implementada uma constante `ALERT_COOLDOWN_MS` (10000ms) que impede que oscilações contínuas do acelerómetro (comuns após uma queda) disparem múltiplos alertas simultâneos para o servidor.

4.1.1 Deteção de Quedas e Acidentes

A deteção de eventos físicos é realizada através do `SensorEventListener` associado ao acelerómetro. A força G é calculada segundo a fórmula:

$$G = \frac{\sqrt{x^2 + y^2 + z^2}}{9.8}$$

Valores superiores a **5G** desencadeiam a lógica de **queda**

Valores superiores a **10G** são interpretados como **acidente**

Este método implementa a interface `SensorEventListener` e é invocado de forma assíncrona pelo sistema sempre que ocorrem alterações nas leituras do acelerômetro. A rotina calcula a magnitude do vetor de aceleração tridimensional (x,y,z)(x, y, z)(x,y,z), normalizando-a para unidades de força G (9,81 m/s2)(9,81 \, m/s^2)(9,81m/s2).

O algoritmo compara o valor resultante com limiares (*thresholds*) pré-definidos, de modo a categorizar a severidade do evento físico detectado: impactos superiores a 5G são classificados como *Queda*, enquanto valores superiores a 10G desencadeiam o protocolo de *Acidente*. Adicionalmente, o método atualiza o *timestamp* da variável de estado `lastMovementTime`, que funciona como *heartbeat* para a lógica de detecção de inatividade.

```
override fun onSensorChanged(event: SensorEvent?) {
    event?.let {
        if (it.sensor.type == Sensor.TYPE_ACCELEROMETER) {
            val x = it.values[0]
            val y = it.values[1]
            val z = it.values[2]
            val gForce = sqrt( x = (x * x + y * y + z * z).toDouble()) / 9.81

            if (abs( x = gForce - 1.0) > 0.1) lastMovementTime = System.currentTimeMillis()

            if (gForce > 3) {
                val timeDiff = System.currentTimeMillis() - lastAlertTime
                if (timeDiff < ALERT_COOLDOWN_MS) return
                val monitors = getMonitorsForRule( type = RuleType.FALL)
                if (monitors.isNotEmpty()) {
                    Log.i(TAG,  msg = "IMPACTO ($gForce G) -> ALERTA")
                    triggerAlert( type = RuleType.FALL,  targetMonitors = monitors)
                }
            }
            if (gForce > 5.0) {
                val timeDiff = System.currentTimeMillis() - lastAlertTime
                if (timeDiff > ALERT_COOLDOWN_MS) {
                    val monitors = getMonitorsForRule( type = RuleType.ACCIDENT)
                    if (monitors.isNotEmpty()) {
                        triggerAlert( type = RuleType.ACCIDENT,  targetMonitors = monitors)
```

4.1.2 Geofencing e Velocidade

Método associado ao `FusedLocationProviderClient`, executado aquando da receção de novos objetos `Location`. A função itera sobre o conjunto de regras de segurança ativas, validando em tempo real restrições de natureza espacial e cinemática:

- **Velocidade:** compara a velocidade instantânea do dispositivo (convertida para km/h) com o limite máximo parametrizado pelo Monitor.
- **Geofencing:** calcula a distância geodésica entre a posição atual e o centro da zona segura definida, recorrendo ao método `Location.distanceBetween`, verificando se o raio de segurança foi ultrapassado.

A violação de qualquer uma destas condições resulta no registo imediato de um alerta.

```
private fun processLocation(location: Location) {
    if (!areRulesLoaded) return

    val speedRules = activeRules.filter { it.type == RuleType.SPEED && isRuleActiveNow( rule = it) }
    val speedTriggeredMonitors = ArrayList<String>()
    speedRules.forEach { rule ->
        val maxSpeed = rule.parameters["value"]?.toFloatOrNull() ?: 100f
        if ((location.speed * 3.6f) > maxSpeed) {
            if(rule.monitorId.isNotBlank()) speedTriggeredMonitors.add(rule.monitorId)
        }
    }
    if (speedTriggeredMonitors.isNotEmpty()) {
        val timeDiff = System.currentTimeMillis() - lastAlertTime
        if (timeDiff > ALERT_COOLDOWN_MS) {
            triggerAlert( type = RuleType.SPEED,  targetMonitors = ArrayList( c = speedTriggeredMonitors.distinct()))
            return
        }
    }

    val geoRules = activeRules.filter { it.type == RuleType.GEOFENCING && isRuleActiveNow( rule = it) }
    val geoTriggeredMonitors = ArrayList<String>()
    geoRules.forEach { rule ->
        val centerLat = rule.parameters["latitude"]?.toDoubleOrNull()
        val centerLon = rule.parameters["longitude"]?.toDoubleOrNull()
        val radius = rule.parameters["radius"]?.toFloatOrNull() ?: 500f
        if (centerLat != null && centerLon != null) {
            val results = FloatArray( size = 1)
            Location.distanceBetween( startLatitude = location.latitude,  startLongitude = location.longitude,  endLatitude = centerLat,  endLor
            val distanceInMeters = results[0]
            if (distanceInMeters > radius) {
                if (rule.monitorId.isNotBlank()) geoTriggeredMonitors.add(rule.monitorId)
            }
        }
    }
    if (geoTriggeredMonitors.isNotEmpty()) {
        val timeDiff = System.currentTimeMillis() - lastAlertTime
        if (timeDiff > ALERT_COOLDOWN_MS) {
            triggerAlert( type = RuleType.GEOFENCING,  targetMonitors = ArrayList( c = geoTriggeredMonitors.distinct()))
        }
    }
```

4.1.3 Verificação de Janelas Temporais

Função auxiliar determinística responsável pela validação do contexto temporal das regras de segurança. O método verifica a integridade lógica da aplicação da regra, assegurando que esta se encontra ativa (`isActive`) e, caso o agendamento esteja habilitado, que o instante atual interseta a janela temporal configurada.

A validação recorre à classe `Calendar` para discretizar o tempo corrente em dias da semana e minutos decorridos desde a meia-noite, comparando estes valores com os intervalos de início e fim definidos nos parâmetros da regra.

```
private fun isRuleActiveNow(rule: SafetyRule): Boolean {
    if (!rule.isActive) return false
    val scheduleEnabled = rule.parameters["scheduleEnabled"] == "true"
    if (!scheduleEnabled) return true

    val cal = Calendar.getInstance()
    val currentDay = cal.get(Calendar.DAY_OF_WEEK)
    val allowedDays = rule.parameters["days"] ?: ""
    if (allowedDays.isNotBlank()) {
        val daysList = allowedDays.split( ...delimiters = ",").mapNotNull { it.toIntOrNull() }
        if (!daysList.contains(currentDay)) return false
    }

    val startTimeStr = rule.parameters["startTime"] ?: "00:00"
    val endTimeStr = rule.parameters["endTime"] ?: "23:59"
    val currentHour = cal.get(Calendar.HOUR_OF_DAY)
    val currentMinute = cal.get(Calendar.MINUTE)
    val currentTimeVal = currentHour * 60 + currentMinute
    val (startH, startM) = parseTime(startTimeStr)
    val (endH, endM) = parseTime(endTimeStr)
    val startTimeVal = startH * 60 + startM
    val endTimeVal = endH * 60 + endM

    return if (startTimeVal <= endTimeVal) {
        currentTimeVal in startTimeVal ≤ .. ≤ endTimeVal
    } else {
        currentTimeVal !in (endTimeVal + 1) ≤ ..<startTimeVal
    }
}
```

4.1.4 Deteção de Inatividade

Inicializa um mecanismo de *polling* recursivo gerido por um `Handler` associado à *Main Thread*. Esta rotina instancia um `Runnable` responsável por monitorizar o estado passivo do utilizador, calculando o delta temporal entre o instante atual e o último registo de movimento significativo (`lastMovementTime`).

Caso este intervalo exceda a duração limite estipulada pelas regras de inatividade ativas, o sistema infere uma situação de risco (por exemplo, inconsciência ou imobilidade prolongada) e emite o respetivo alerta. A verificação é reagendada ciclicamente, garantindo a monitorização contínua do utilizador.

```
private fun startInactivityCheck() {
    inactivityRunnable = object : Runnable {
        override fun run() {
            val inactiveTime = System.currentTimeMillis() - lastMovementTime
            val triggeredMonitors = ArrayList<String>()
            activeRules.filter { it.type == RuleType.INACTIVITY && isRuleActiveNow( rule = it) }.forEach { rule ->
                val maxDur = (rule.parameters["duration"]?.toLongOrNull() ?: 60L) * 60 * 1000
                if (inactiveTime > maxDur) {
                    if(rule.monitorId.isNotBlank()) triggeredMonitors.add(rule.monitorId)
                }
            }
            if (triggeredMonitors.isNotEmpty()) {
                lastMovementTime = System.currentTimeMillis()
                triggerAlert( type = RuleType.INACTIVITY,  targetMonitors = ArrayList( c = triggeredMonitors.distinct()))
            }
            inactivityHandler.postDelayed( r = this,  delayMillis = 30000L)
        }
    }
    inactivityHandler.post( r = inactivityRunnable!!)
}
```

4.2. Captura de Prova em Vídeo (CameraX)

Para garantir a recolha de evidências forenses após a deteção de um evento crítico, a aplicação integra a API **CameraX** do Android Jetpack, cuja lógica se encontra encapsulada na classe utilitária `VideoManager`.

O processo de gravação é iniciado automaticamente durante o ciclo de vida da `AlertActivity`, sendo configurado para capturar clipes de 30 segundos com definições de qualidade otimizadas para transmissão rápida (`Quality.LOWEST`). O ficheiro resultante é armazenado temporariamente no diretório de cache da aplicação (`context.cacheDir`), assegurando que não ocupa armazenamento permanente desnecessário no dispositivo.

A persistência dos dados segue um fluxo assíncrono e condicional:

- Caso o alerta não seja cancelado (através da validação do PIN), o vídeo é submetido para o **Firebase Storage**, no caminho `alert_videos/`.
- Após a conclusão bem-sucedida do upload, o URL de download retornado é imediatamente injetado no documento do alerta correspondente, na coleção `alerts` do **Firestore**, permitindo que o Monitor visualize a ocorrência em tempo real através do componente `VideoPlayerDialog`.

4.3. Associação Segura e Autenticação

Autenticação Multifator (MFA)

De forma a reforçar a segurança do processo de autenticação, foi implementado um mecanismo de **Autenticação Multifator (MFA)** na camada `AuthRepository`. Após a validação das credenciais primárias (email/password), o sistema gera um código numérico aleatório de cinco dígitos, que é armazenado temporariamente na coleção `mfa_codes`.

Na versão atual, para efeitos de depuração e demonstração, este código é emitido através do **Logcat**, simulando o comportamento de um serviço externo de envio por SMS ou gateway de e-mail.

Associação Monitor–Protegido
O estabelecimento de uma relação de confiança entre utilizadores é realizado através de um protocolo de *handshake* seguro, gerido pela classe `AssociationRepository`:

- O utilizador do tipo **Protegido** gera um token alfanumérico efémero (seis caracteres), que é armazenado na coleção `temp_codes`, juntamente com um *timestamp* de criação.

- Este token possui uma validade estrita de cinco minutos, verificada no momento da utilização, mitigando riscos de reutilização indevida.

- Quando o Monitor submete um código válido, é executada uma transação atómica em *batch* (`db.batch()`) no Firestore. Esta operação garante a consistência dos dados ao atualizar, em simultâneo, o array `monitoring` no perfil do Monitor e o array `monitoredBy` no perfil do Protegido, removendo o código temporário na mesma transação.

4.4. Gestão de Alertas e Notificações

Cancelamento Seguro
A interrupção de um alerta ativo na `AlertActivity` requer prova de identidade através de um PIN de segurança (campo `cancelCode` do utilizador). Esta salvaguarda previne cancelamentos acidentais ou realizados sob coação por terceiros.

Sistema de Notificação em Tempo Real
A arquitetura de notificações privilegia a reatividade imediata através de dois mecanismos complementares:

- **Monitorização em Primeiro Plano (Foreground):** a interface do Monitor subscreve diretamente as alterações na coleção `alerts` através de um `SnapshotListener`, permitindo a atualização instantânea da interface gráfica.

- **Serviço em Segundo Plano (Background):** o `MonitorService` opera como um *Foreground Service* Android, mantendo uma escuta ativa (`addSnapshotListener`) sobre a base de dados mesmo quando a aplicação se encontra encerrada. Ao detetar um novo documento com o estado `ACTIVE`, direcionado ao Monitor, o serviço emite uma Notificação de Sistema de alta prioridade (`NotificationCompat`), assegurando que o evento crítico não passa despercebido.

5. Estrutura de Dados (Firestore)

A base de dados foi modelada com foco na eficiência e escalabilidade:

- **users (Collection)**
  - Campos: `uid`, `name`, `email`, `cancelCode`
  - Arrays: `monitoring`, `monitoredBy`

```
data class SafetyUser(
    val id: String = "",
    val email: String = "",
    val name: String = "",
    val cancelCode: String = "0000",


    val monitoring: List<String> = emptyList(),


    val monitoredBy: List<String> = emptyList(),
)
```

- **users/{uid}/rules (Subcoleção)**
  - Campos: `type` (FALL, SPEED, etc.), `isActive`, `parameters` (mapa com raio, velocidade, horários)

```
enum class RuleType {
    7 Usages
    FALL,
    2 Usages
    ACCIDENT,
    5 Usages
    GEOFENCING,
    5 Usages
    SPEED,
    5 Usages
    INACTIVITY,
    3 Usages
    PANIC_BUTTON
}

16 Usages
data class SafetyRule(
    var id: String = "",

    var monitorId: String = "",

    var type: RuleType = RuleType.FALL,

    @get:PropertyName( value = "isActive")
    @set:PropertyName( value = "isActive")
    var isActive: Boolean = true,

    var parameters: Map<String, String> = emptyMap()
)
```

- **alerts (Collection)**
    - Campos: `monitorId`, `type`, `coordinates`, `videoUrl`, `status`, `timestamp`

```
data class SafetyAlert(
    val id: String = "",
    val monitorId: String = "",
    val type: RuleType = RuleType.PANIC_BUTTON,
    val protectedId: String = "",
    val protectedName: String = "",
    val coordinates: String = "",
    val timestamp: Timestamp = Timestamp.now(),
    val videoUrl: String? = null,
    val status: String = "ACTIVE"
)
```

- **temp_codes / mfa_codes**
    - Coleções temporárias para gestão de códigos de associação e autenticação

```
data class MfaCode(
    val code: String = "",
    val email: String = "",
    val timestamp: Long = 0
)
```

6. Interface e Usabilidade (UI/UX)

A interface foi concebida com especial atenção à acessibilidade e adaptabilidade:

- **Internacionalização:**
  Suporte completo para Português e Inglês, com mudança dinâmica de idioma sem reinício da aplicação, utilizando `ContextWrapper` personalizados.

- **Orientação e Responsividade:**
  Todos os ecrãs suportam corretamente os modos *Portrait* e *Landscape*, recorrendo a `verticalScroll`, `BoxWithConstraints` e layouts flexíveis com pesos.

- **Feedback ao Utilizador:**
  Utilização consistente de diálogos de erro, indicadores de progresso (`CircularProgressIndicator`) e *Toasts* para informar o estado das operações críticas, como uploads de vídeo ou obtenção de localização.

7. Conclusão e Trabalho Futuro

Apesar da robustez da solução, foram identificadas várias oportunidades de evolução:

Autenticação Multifactor Real
Substituição da simulação actual por uma integração efectiva com SMS, correio electrónico ou aplicações autenticadoras, aumentando o nível de segurança em ambiente de produção.

Optimização Energética
Implementação de estratégias adaptativas de monitorização, ajustando a frequência de utilização dos sensores e da localização de acordo com o contexto (por exemplo, períodos prolongados de imobilidade).

Acessibilidade Avançada
Melhor integração com leitores de ecrã (TalkBack), *feedback* háptico configurável e interfaces específicas para utilizadores com limitações motoras ou cognitivas.

Escalabilidade do Back-end
Evolução da modelação de dados de forma a suportar um maior número de associações e históricos de alertas, recorrendo a indexação avançada e a funções *serverless* (Cloud Functions).

As soluções técnicas adoptadas demonstram uma aplicação consistente de boas práticas de desenvolvimento móvel moderno, equilibrando funcionalidade, segurança e experiência de utilização. O projecto constitui uma base sólida para evolução futura, quer em contexto académico, quer como protótipo funcional com potencial de aplicação em cenários reais.